

Generowanie szumu Perlina

Implementacja i optymalizacja na CPU oraz GPU (CUDA)

Dokumentacja techniczna projektu

Język implementacji:	C++17, CUDA C++
Kompilator GPU:	NVCC (CUDA Toolkit)
Standard C++:	C++17
Wyjście:	obrazy PGM, pliki CSV z wynikami

Spis treści

1	Wprowadzenie	3
2	Model matematyczny szumu Perlina	3
2.1	Podstawa działania	3
2.2	Szum gradientowy	3
2.3	Wyznaczenie komórki siatki	4
2.4	Tabela permutacji	4
2.5	Wektory gradientów	5
2.6	Iloczynny skalarne dla narożników komórki	5
2.7	Funkcja wygładzająca	6
2.8	Interpolacja liniowa	6
2.9	Zakres wartości funkcji	7
2.10	Fractal Brownian Motion (fBm)	8
2.11	Podsumowanie	9
3	Implementacja algorytmu szumu Perlina dla CPU	9
3.1	Struktura implementacji	9
3.2	Inicjalizacja tablicy permutacji	9
3.3	Funkcja wygładzająca	10
3.4	Interpolacja liniowa	10
3.5	Wyznaczanie gradientów	11
3.6	Generowanie pojedynczej wartości szumu Perlina	11
3.7	Implementacja funkcji fBm	13
3.8	Generowanie obrazu szumu	14
3.9	Pomiar czasu wykonania	15
4	Implementacja algorytmu szumu Perlina dla GPU z wykorzystaniem CUDA	15
4.1	Przeniesienie algorytmu z CPU na GPU	15
4.2	Przechowywanie tablicy permutacji w pamięci stałej	16
4.3	Inicjalizacja tablicy permutacji	16
4.4	Funkcje urządzenia	17
4.5	Implementacja funkcji <code>perlin_device()</code>	17
4.6	Implementacja funkcji fBm	18
4.7	Kernel CUDA	18
4.8	Konfiguracja siatki wątków	19
4.9	Uruchomienie kernela	19
5	Optymalizacja implementacji CUDA	20
5.1	Zakres wprowadzonych optymalizacji	20
5.2	Eliminacja narzutu wywołań funkcji	20
5.3	Optymalizacja obliczania części całkowitej współrzędnych	21
5.4	Zastosowanie kwalifikatora <code>restrict</code>	22
5.5	Porównanie funkcji <code>perlin_device()</code> i <code>perlin_opt()</code>	22
5.6	Porównanie funkcji <code>fbm_device()</code> i <code>fbm_opt()</code>	23
5.7	Wpływ optymalizacji na wydajność	23

6	Obsługa benchmarków oraz zapisu wyników	23
6.1	Program główny	23
6.2	Moduł benchmarków	24
6.3	Zapis obrazów	24
6.4	Eksport wyników do pliku CSV	25
7	Analiza wyników i wnioski	26
7.1	Wnioski	29

1 Wprowadzenie

Szum Perlina (*Perlin Noise*) jest funkcją szumu gradientowego stworzona przez Kena Perlina w 1982 roku w celu generowania naturalnie wyglądających tekstur i efektów proceduralnych w grafice komputerowej. W przeciwieństwie do klasycznego szumu losowego (*white noise*), który cechował się nagłymi i ostrymi zmianami wartości pomiędzy sąsiednimi punktami, szum Perlina generuje przejścia bardziej ciągłe i gładkie.

Jest stosowany między innymi w:

- generowaniu map wysokości terenu,
- symulacji ognia, chmur i dymu,
- tworzeniu tekstur drewna, marmuru oraz skał,
- proceduralnej generacji światów w grach komputerowych,

2 Model matematyczny szumu Perlina

2.1 Podstawa działania

Podstawą algorytmu jest rozmieszczenie pseudolosowych wektorów gradientów w punktach regularnej siatki, a następnie płynne interpolowanie ich wpływu pomiędzy punktami w tej przestrzeni.

2.2 Szum gradientowy

Szum Perlina należy do klasy szumów gradientowych.

Dla dwuwymiarowej przestrzeni $\mathbb{R}^2$. W każdym węźle siatki definiowany jest pseudolosowy wektor gradientu:

$$\mathbf{g}_{ij}, \tag{1}$$

gdzie

$$i, j \in \mathbb{Z}. \tag{2}$$

Dla dowolnego punktu

$$P = (x, y) \tag{3}$$

algorytm określa komórkę siatki, w której znajduje się punkt, a następnie oblicza wpływ najbliższych narożników tej komórki, co pozwala to uzyskać znacznie bardziej naturalne przejścia pomiędzy kolejnymi obszarami przestrzeni.

2.3 Wyznaczenie komórki siatki

Dla punktu (x, y) wyznaczone są współrzędne całkowite:

$$X = \lfloor x \rfloor, \quad (4)$$

$$Y = \lfloor y \rfloor, \quad (5)$$

oraz części ułamkowe:

$$x_f = x - \lfloor x \rfloor, \quad (6)$$

$$y_f = y - \lfloor y \rfloor. \quad (7)$$

Wartości X oraz Y oznaczają komórkę siatki, a współrzędne (x_f, y_f) określają lokalne położenie punktu wewnątrz tej komórki.

Po wykonaniu tego kroku dalsze obliczenia prowadzone są w lokalnym układzie współrzędnych:

$$(x_f, y_f) \in [0, 1]^2. \quad (8)$$

2.4 Tabela permutacji

Ważnym elementem algorytmu jest predefiniowana tabela permutacji wykorzystywana jako funkcja haszująca.

Ma ona rozmiar 256 elementów i jest zbudowana z permutacji liczb:

$$0, 1, 2, \dots, 255. \quad (9)$$

Tablica ta jest zwykle duplikowana do rozmiaru 512 elementów:

$$P[256 + i] = P[i]. \quad (10)$$

Pozwala to uniknąć dodatkowych operacji modulo podczas indeksowania.

Tabela permutacji zapewnia:

- deterministyczność wyników,
- pseudolosowy dobór gradientów,
- możliwość powtarzalnego generowania identycznych struktur.

2.5 Wektory gradientów

Po wyznaczeniu pseudolosowego indeksu za pomocą tablicy permutacji wybierany jest odpowiadający mu wektor gradientu.

W przedstawionej implementacji liczba możliwych gradientów została ograniczona do ośmiu. Wybór konkretnego gradientu realizowany jest za pomocą operacji bitowej:

$$h = \text{hash} \& 7, \quad (11)$$

która wykorzystuje trzy najmłodsze bity wartości uzyskanej z tablicy permutacji. Pozwala to uzyskać osiem różnych konfiguracji znaków i orientacji gradientów bez konieczności przechowywania dodatkowej tablicy wektorów, co zmniejsza liczbę odwołań do pamięci.

Dla każdego narożnika obliczany jest wektor przesunięcia od narożnika do analizowanego punktu:

$$\mathbf{d}. \quad (12)$$

Następnie wyznaczany jest iloczyn skalarny:

$$s = \mathbf{g} \cdot \mathbf{d}, \quad (13)$$

gdzie:

- $\mathbf{g}$ – wektor gradientu,
- $\mathbf{d}$ – wektor przesunięcia.

Otrzymana wartość określa wpływ danego narożnika na końcowy wynik funkcji szumu.

2.6 Iloczyny skalarne dla narożników komórki

W przypadku dwuwymiarowym rozpatrywane są cztery narożniki komórki:

$$(X, Y), \quad (14)$$

$$(X + 1, Y), \quad (15)$$

$$(X, Y + 1), \quad (16)$$

$$(X + 1, Y + 1). \quad (17)$$

Dla każdego z nich wyznaczana jest wartość:

$$n_{00}, \quad (18)$$

$$n_{10}, \quad (19)$$

$$n_{01}, \quad (20)$$

$$n_{11}. \quad (21)$$

Są to wyniki iloczynów skalarnych pomiędzy gradientami i odpowiednimi wektorami przesunięcia.

Wartości te są podstawę do dalszego etapu interpolacji.

2.7 Funkcja wygładzająca

Aby wyeliminować widoczne ostre granice pomiędzy sąsiednimi komórkami siatki, Ken Perlin zaproponował funkcję wygładzającą:

$$f(t) = 6t^5 - 15t^4 + 10t^3. \quad (22)$$

Funkcja ta spełnia następujące warunki:

$$f(0) = 0, \quad (23)$$

$$f(1) = 1, \quad (24)$$

$$f'(0) = 0, \quad (25)$$

$$f'(1) = 0, \quad (26)$$

$$f''(0) = 0, \quad (27)$$

$$f''(1) = 0. \quad (28)$$

Zerowe wartości pierwszej i drugiej pochodnej na krańcach przedziału powodują, że przejścia pomiędzy komórkami są bardzo płynne i pozbawione artefaktów.

2.8 Interpolacja liniowa

Po obliczeniu wartości dla czterech narożników komórki siatki konieczne jest wyznaczenie pojedynczej wartości szumu dla analizowanego punktu. W tym celu stosowana jest interpolacja, która pozwala płynnie przechodzić pomiędzy wartościami wyznaczonymi w narożnikach komórki.

Położenie punktu wewnątrz komórki opisują części ułamkowe współrzędnych:

$$x_f = x - \lfloor x \rfloor, \quad (29)$$

$$y_f = y - \lfloor y \rfloor. \quad (30)$$

Na ich podstawie obliczane są wagi interpolacji:

$$u = fade(x_f), \quad (31)$$

$$v = fade(y_f). \quad (32)$$

Wartości u oraz v określają, jak duży wpływ na wynik mają poszczególne narożniki komórki. Dla punktów znajdujących się bliżej lewej krawędzi większy wpływ mają wartości pochodzące z lewych narożników, natomiast dla punktów położonych bliżej prawej krawędzi większe znaczenie mają narożniki prawe. Analogiczna zależność występuje w kierunku pionowym.

Interpolacja liniowa realizowana jest za pomocą funkcji:

$$\text{lerp}(a, b, t) = a + t(b - a), \quad (33)$$

gdzie:

- a i b są interpolowanymi wartościami,
- $t \in [0, 1]$ jest współczynnikiem określającym położenie pomiędzy nimi.

Najpierw wykonywana jest interpolacja pomiędzy wartościami dolnych narożników komórki:

$$i_1 = \text{lerp}(n_{00}, n_{10}, u), \quad (34)$$

oraz pomiędzy wartościami górnych narożników:

$$i_2 = \text{lerp}(n_{01}, n_{11}, u). \quad (35)$$

W rezultacie otrzymywane są dwie wartości odpowiadające położeniu punktu wzdłuż osi x .

Następnie wykonywana jest druga interpolacja, tym razem wzdłuż osi y :

$$N(x, y) = \text{lerp}(i_1, i_2, v). \quad (36)$$

Otrzymana wartość $N(x, y)$ stanowi końcowy wynik szumu Perlina dla punktu (x, y) .

Proces ten jest w zasadzie dwuetapowym mieszaniem wpływu czterech narożników komórki. Dzięki zastosowaniu wag u i v , wyznaczonych za pomocą funkcji wygładzającej, przejścia pomiędzy sąsiednimi komórkami pozostają płynne, a wygenerowany szum nie zawiera widocznych granic siatki.

2.9 Zakres wartości funkcji

Wartość klasycznego szumu Perlina przyjmuje wartości zbliżone do przedziału:

$$N(x, y) \in [-1, 1]. \quad (37)$$

W celu uzyskania wartości z zakresu $[0, 1]$ stosowana jest normalizacja:

$$N' = \frac{N + 1}{2}. \quad (38)$$

Jeżeli wynik ma zostać zapisany jako obraz w skali szarości, wykonywane jest dodatkowe przeskalowanie:

$$G = 255 \cdot N'. \quad (39)$$

2.10 Fractal Brownian Motion (fBm)

Pojedyncza warstwa szumu Perlina zawiera ograniczoną ilość szczegółów. Z tego powodu w praktyce wykorzystuje się technikę *Fractal Brownian Motion* (fBm), polegającą na sumowaniu wielu warstw szumu o różnych częstotliwościach i amplitudach.

Funkcja fBm opisana jest wzorem:

$$fBm(x, y) = \sum_{i=0}^{n-1} A_i N(F_i x, F_i y), \quad (40)$$

gdzie:

- A_i – amplituda i -tej oktawy,
- F_i – częstotliwość i -tej oktawy,
- n – liczba oktaw.

Oktawa oznacza pojedynczą warstwę szumu wykorzystywaną podczas obliczania funkcji fBm.

Amplituda określa siłę wpływu danej oktawy na końcowy wynik funkcji. Większa amplituda powoduje większy udział danej warstwy szumu w generowanym obrazie, natomiast mniejsza amplituda ogranicza jej wpływ.

Częstotliwość określa skalę generowanych struktur. Niskie częstotliwości odpowiadają dużym, łagodnie zmieniającym się elementom, natomiast wysokie częstotliwości generują drobniejsze szczegóły i bardziej dynamiczne zmiany wartości szumu.

W praktyce pierwsze oktawy odpowiadają za ogólny kształt generowanej struktury, natomiast kolejne oktawy stopniowo dodają coraz mniejsze detale.

Na przykład podczas generowania map wysokości terenu pierwsze oktawy tworzą główne pasma górskie i doliny, natomiast kolejne oktawy odpowiadają za mniejsze wzniesienia, uskoki i lokalne nierówności powierzchni.

Kolejne amplitudy oraz częstotliwości wyznaczane są zgodnie z zależnościami:

$$A_{i+1} = A_i \cdot persistence, \quad (41)$$

$$F_{i+1} = F_i \cdot lacunarity. \quad (42)$$

Parametr *persistence* kontroluje tempo zanikania amplitudy kolejnych oktaw, natomiast *lacunarity* odpowiada za wzrost częstotliwości.

Dzięki zastosowaniu wielu oktafów możliwe jest generowanie wieloskalowych struktur przypominających rzeczywiste krajobrazy, chmury oraz powierzchnie naturalne.

2.11 Podsumowanie

Model matematyczny szumu Perlina można podzielić na cztery podstawowe etapy:

1. wyznaczenia komórki siatki dla punktu wejściowego,
2. przypisania gradientów za pomocą tabeli permutacji,
3. obliczenia iloczynów skalarnych pomiędzy gradientami i wektorami przesunięcia,
4. interpolacji wyników przy użyciu funkcji wygładzającej.

Otrzymana funkcja jest ciągła oraz posiada gładkie przejścia pomiędzy sąsiednimi obszarami przestrzeni. Rozszerzenie algorytmu o metodę fBm pozwala generować złożone, realistyczne struktury wykorzystywane we współczesnej grafice proceduralnej.

3 Implementacja algorytmu szumu Perlina dla CPU

Celem implementacji jest wygenerowanie dwuwymiarowej mapy szumu zapisanej w postaci obrazu w skali szarości.

Kod źródłowy został podzielony na kilka funkcji realizujących poszczególne etapy algorytmu. Dzięki temu struktura programu odpowiada bezpośrednio kolejnym krokom modelu matematycznego przedstawionego wcześniej.

3.1 Struktura implementacji

Implementacja składa się z:

- inicjalizacji tablicy permutacji,
- funkcji wygładzającej `fade()`,
- funkcji interpolacji liniowej `lerp()`,
- funkcji wyznaczającej gradient `grad()`,
- funkcji generującej pojedynczą wartość szumu Perlina `perlin()`,
- funkcji realizującej Fractal Brownian Motion `fbm()`,
- funkcji generującej kompletny obraz szumu `generateNoiseCPU()`.

3.2 Inicjalizacja tablicy permutacji

Pierwszym etapem działania algorytmu jest przygotowanie tablicy permutacji wykorzystywanej podczas wyboru gradientów.

```
1 static bool initialized = false;  
2 static int permutationTable[512];
```

Tablica `permutationTable` posiada 512 elementów. Pierwsze 256 pozycji zawiera wartości z podstawowej tablicy permutacji, natomiast kolejne 256 elementów stanowi jej kopię.

Inicjalizacja wykonywana jest tylko raz podczas pierwszego wywołania funkcji:

```
1 static void initPermutation()
2 {
3     if (initialized)
4     {
5         return;
6     }
7
8     for (int i = 0; i < 256; i++)
9     {
10         permutationTable[i] = basePermutation[i];
11         permutationTable[256 + i] = basePermutation[i];
12     }
13
14     initialized = true;
15 }
```

Duplikacja tablicy pozwala na indeksowanie bez konieczności stosowania dodatkowych operacji modulo, co upraszcza kod i poprawia wydajność.

3.3 Funkcja wygładzająca

Implementacja funkcji wygładzającej odpowiada bezpośrednio wzorowi matematycznemu przedstawionemu wcześniej:

$$f(t) = 6t^5 - 15t^4 + 10t^3. \quad (43)$$

W kodzie funkcja została zaimplementowana następująco:

```
1 static float fade(float t)
2 {
3     return t * t * t * (t * (t * 6 - 15) + 10);
4 }
```

Zastosowanie wyłącznie operacji mnożenia pozwala ograniczyć liczbę kosztownych obliczeń oraz zwiększa wydajność wykonywania funkcji.

Funkcja `fade()` odpowiada za wygładzenie wag interpolacji i eliminuje widoczne granice pomiędzy sąsiednimi komórkami siatki.

3.4 Interpolacja liniowa

Interpolacja liniowa została zaimplementowana za pomocą funkcji:

```
1 static float lerp(float a, float b, float t)
2 {
3     return a + t * (b - a);
4 }
```

Realizuje ona zależność:

$$\text{lerp}(a, b, t) = a + t(b - a). \quad (44)$$

Funkcja wykorzystywana jest podczas obliczania końcowej wartości szumu poprzez płynne łączenie wpływu sąsiednich narożników komórki.

3.5 Wyznaczanie gradientów

Kolejnym elementem implementacji jest funkcja odpowiedzialna za wybór gradientu.

```

1 static float grad(int hash,
2                   float x,
3                   float y)
4 {
5     int h = hash & 7;
6
7     float u = h < 4 ? x : y;
8     float v = h < 4 ? y : x;
9
10    return ((h & 1) ? -u : u)
11           + ((h & 2) ? -v : v);
12 }
```

Parametr `hash` pochodzi z tablicy permutacji i określa, który gradient zostanie wykorzystany.

Instrukcja:

```
1 int h = hash & 7;
```

pozostawia trzy najmłodsze bity liczby, co daje osiem możliwych kombinacji:

$$2^3 = 8. \quad (45)$$

Dzięki temu implementacja wykorzystuje osiem możliwych kierunków gradientów bez konieczności przechowywania dodatkowej tablicy wektorów.

Wynikiem funkcji jest iloczyn skalarny gradientu oraz wektora przesunięcia od narożnika komórki do analizowanego punktu.

3.6 Generowanie pojedynczej wartości szumu Perlina

Najważniejszym elementem implementacji jest funkcja obliczająca pojedyncze próbki szumu Perlina.:

```
1 float perlin(float x, float y)
```

Na początku wytyczone zostają współrzędne komórki siatki wyznaczane są za pomocą instrukcji:

```

1 int X = (int)floor(x) & 255;
2 int Y = (int)floor(y) & 255;
```

Funkcja `floor()` zwraca największą liczbę całkowitą nie większą od podanej wartości. Dzięki temu dostajemy współrzędne narożnika komórki, w której znajduje się analizowany punkt.

Przykładowo dla punktu:

$$(x, y) = (12.73, 5.41) \quad (46)$$

otrzymujemy:

$$X = 12, \quad (47)$$

$$Y = 5. \quad (48)$$

Operacja bitowa:

$$\& 255 \quad (49)$$

ogranicza zakres współrzędnych do przedziału od 0 do 255. Jest to równoważne wykonaniu operacji modulo 256, lecz realizowane jest szybciej. Pozwala to poprawnie indeksować tablicę permutacji zawierającą 256 podstawowych elementów oraz zapewnia cykliczne powtarzanie wzorca szumu dla większych współrzędnych.

Następnie obliczane są lokalne współrzędne punktu wewnątrz komórki:

```
1 x -= floor(x);  
2 y -= floor(y);
```

Kolejnym krokiem jest wyznaczenie wag interpolacji:

```
1 float u = fade(x);  
2 float v = fade(y);
```

Po obliczeniu wag interpolacji konieczne jest wyznaczenie gradientów przypisanych do czterech narożników aktualnej komórki siatki. Do tego wykorzystywana jest tablica permutacji pełniąca rolę funkcji haszującej.

```
1 int aa = permutationTable[  
2     permutationTable[X] + Y];  
3  
4 int ab = permutationTable[  
5     permutationTable[X] + Y + 1];  
6  
7 int ba = permutationTable[  
8     permutationTable[X + 1] + Y];  
9  
10 int bb = permutationTable[  
11     permutationTable[X + 1] + Y + 1];
```

Poszczególne zmienne odpowiadają czterem narożnikom komórki:

- **aa** – narożnik (X, Y) ,
- **ab** – narożnik $(X, Y + 1)$,
- **ba** – narożnik $(X + 1, Y)$,
- **bb** – narożnik $(X + 1, Y + 1)$.

Dla każdego narożnika wykonywane są dwa odwołania do tablicy permutacji. Pierwsze odwołanie wykorzystuje współrzędną X , natomiast drugie uwzględnia również współrzędną Y . Dzięki temu każda para współrzędnych siatki jest przekształcana w pseudolosową wartość całkowitą.

Otrzymane wartości nie są jeszcze gradientami, tylko indeksami wykorzystywanymi przez funkcję `grad()` do wyboru odpowiedniego kierunku gradientu. Dzięki temu zapewniamy, że dla tych samych współrzędnych zawsze zostanie wybrany ten sam gradient, zachowując pozorną losowość rozkładu w całej przestrzeni.

Ostatnim etapem jest wykonanie dwuetapowej interpolacji:

```
1 float result =
2     lerp(
3         lerp(
4             grad(aa, x, y),
5             grad(ba, x - 1.0f, y),
6             u),
7         lerp(
8             grad(ab, x, y - 1.0f),
9             grad(bb, x - 1.0f, y - 1.0f),
10            u),
11     v);
```

Najpierw wykonywana jest interpolacja wzdłuż osi x , a następnie wzdłuż osi y , dzięki czemu uzyskamy finalną wartość szumu dla punktu (x, y) .

3.7 Implementacja funkcji fBm

Funkcja fBm odpowiada za generowanie bardziej szczegółowych struktur poprzez sumowanie wielu oktafów szumu Perlina.

```
1 float fbm(float x,
2           float y,
3           int octaves,
4           float persistence,
5           float lacunarity)
```

Dla każdej oktawy wykonywane są następujące operacje:

- wygenerowanie szumu Perlina,
- przemnożenie wyniku przez aktualną amplitudę,
- zwiększenie częstotliwości,
- zmniejszenie amplitudy.

Wykonane jest to przez pętlę:

```

1 for (int i = 0; i < octaves; i++)
2 {
3     sum += perlin(
4         x * frequency,
5         y * frequency) * amplitude;
6
7     maxValue += amplitude;
8
9     amplitude *= persistence;
10    frequency *= lacunarity;
11 }

```

Po zakończeniu obliczeń wynik jest normalizowany:

```

1 return sum / maxValue;

```

Dzięki temu końcowa wartość pozostaje niezależna od liczby wykorzystanych oktafów.

3.8 Generowanie obrazu szumu

Funkcja `generateNoiseCPU()` odpowiada za wygenerowanie pełnej mapy szumu.

Obliczenia wykonywane są dla każdego piksela obrazu jeden po drugim:

```

1 for (int y = 0; y < height; y++)
2 {
3     for (int x = 0; x < width; x++)
4     {
5         ...
6     }
7 }

```

Dla każdego punktu wywoływana jest funkcja:

```

1 float value =
2     fbm(x * scale,
3         y * scale,
4         octaves,
5         persistence,
6         lacunarity);

```

Wartość zwrócona przez `fbm` znajduje się w zakresie:

$$[-1, 1]. \quad (50)$$

Dlatego wykonywana jest normalizacja:

```

1 value = (value + 1.0f) * 0.5f;

```

oraz konwersja do skali szarości:

```

1 int gray = static_cast<int>(value * 255.0f);

```

Po ograniczeniu wyniku do zakresu 0 – 255 wartość zapisywana jest w buforze obrazu:

```

1 output[y * width + x] = static_cast<unsigned char>(gray);

```

3.9 Pomiar czasu wykonania

W celu porównania wydajności implementacji CPU i GPU zmierzono czas generowania szumu.

Pomiar realizowany jest za pomocą biblioteki `<chrono>`:

```
1 auto start = std::chrono::high_resolution_clock::now();
2
3 <generowanie szumu>
4
5 auto end = std::chrono::high_resolution_clock::now();
```

Następnie obliczany jest czas wykonania:

```
1 return std::chrono::duration<double>(end - start).count();
```

Uzyskana wartość otrzymana jest w sekundach.

4 Implementacja algorytmu szumu Perlina dla GPU z wykorzystaniem CUDA

Kolejnym etapem jest przeniesienie algorytmu na GPU. Głównym celem tej modyfikacji było wykorzystanie dużej liczby rdzeni obliczeniowych do równoczesnego generowania wielu pikseli obrazu.

W przeciwieństwie do implementacji CPU, w której kolejne piksele obliczane są jeden po drugim w zagnieżdżonych pętlach, implementacja CUDA pozwala przypisać obliczenie każdego piksela do osobnego wątku GPU. Dzięki temu tysiące punktów obrazu mogą być przetwarzane równolegle.

4.1 Przeniesienie algorytmu z CPU na GPU

Matematyczny model szumu Perlina pozostaje taki sam dla procesora CPU, jak i GPU. Nie zmieniają się:

- funkcja wygładzająca `fade()`,
- interpolacja liniowa `lerp()`,
- sposób wyznaczania gradientów,
- algorytm generowania pojedynczej próbki szumu,
- metoda Fractal Brownian Motion.

Zmienia się natomiast sposób organizacji obliczeń.

W implementacji CPU obraz generowany był za pomocą pętli:

```
1 for(int y=0; y<height; y++)
2 {
3     for(int x=0; x<width; x++)
4     {
5         ...
6     }
```



```
7 }
```

Procesor wykonywał obliczenia kolejnych pikseli jeden po drugim.

W implementacji CUDA zadanie to jest rozłożone na tysiące równoległe wykonywanych wątków. Każdy wątek oblicza wartość szumu dla dokładnie jednego piksela obrazu.

Obliczenie wartości szumu dla pojedynczego punktu nie wymaga informacji o wynikach obliczeń innych punktów, więc algorytm generowania szumu Perlina bardzo dobrze skaluje się w architekturze równoległej.

4.2 Przechowywanie tablicy permutacji w pamięci stałej

Jednym z najczęściej wykorzystywanych elementów algorytmu jest tablica permutacji.

W implementacji CPU była ona przechowywana w pamięci operacyjnej programu:

```
1 static int permutationTable[512];
```

W wersji CUDA wykorzystano pamięć stałą GPU:

```
1 __constant__ int d_permutation[512];
```

Pamięć stała (*constant memory*) jest specjalnym obszarem pamięci urządzenia przeznaczonym do przechowywania danych odczytywanych wielokrotnie przez dużą liczbę wątków.

Ponieważ wszystkie wątki korzystają z tej samej tablicy permutacji, umieszczenie jej w pamięci stałej pozwala ograniczyć liczbę odwołań do wolniejszej pamięci globalnej GPU.

4.3 Inicjalizacja tablicy permutacji

Przed rozpoczęciem obliczeń potrzebne jest skopiowanie tablicy permutacji z pamięci hosta (CPU) do pamięci urządzenia (GPU).

Realizuje to funkcja:

```
1 void initPermutationGPU()
```

Tworzona jest lokalna kopia tablicy:

```
1 int h_permutation[512];
```

następnie wykonywana jest jej duplikacja:

```
1 for(int i=0; i<256; i++)
2 {
3     h_permutation[i] = basePermutation[i];
4     h_permutation[i+256] = basePermutation[i];
5 }
```

a na końcu kopiowanie do pamięci urządzenia:

```
1 cudaMemcpyToSymbol(
2     d_permutation,
3     h_permutation,
4     sizeof(h_permutation));
```

Operacja ta wykonywana jest tylko raz podczas pierwszego uruchomienia algorytmu.

4.4 Funkcje urządzenia

W implementacji CUDA funkcje wykorzystywane przez GPU oznaczane są kwalifikatorem:

```
1 __device__
```

Przykładowo:

```
1 __device__ float fade(float t)
```

oznacza funkcję wykonywaną na GPU, która może być wywoływana wyłącznie przez inne funkcje działające na urządzeniu.

W analogiczny sposób zaimplementowano funkcje:

- `fade()`,
- `lerp()`,
- `grad()`,
- `perlin_device()`,
- `fbm_device()`.

Ich logika pozostaje praktycznie identyczna jak w implementacji CPU.

4.5 Implementacja funkcji `perlin_device()`

Funkcja:

```
1 __device__  
2 float perlin_device(float x, float y)
```

stanowi bezpośredni odpowiednik funkcji `perlin()` z wersji CPU.

Realizowane są dokładnie te same kroki:

1. wyznaczenie komórki siatki,
2. obliczenie współrzędnych lokalnych,
3. wyznaczenie wag interpolacji,
4. pobranie gradientów,
5. wykonanie interpolacji.

Przykładowo współrzędne komórki obliczane są jako:

```
1 int X = ((int)floorf(x)) & 255;  
2 int Y = ((int)floorf(y)) & 255;
```

a gradienty pobierane są z tablicy permutacji znajdującej się już w pamięci GPU:

```
1 int aa =  
2 d_permutation[d_permutation[X] + Y];
```

Dzięki temu nie ma konieczności komunikacji z procesorem podczas wykonywania obliczeń.

4.6 Implementacja funkcji fBm

Funkcja:

```
1 __device__  
2 float fbm_device(...)
```

jest bezpośrednim odpowiednikiem funkcji CPU.

Dla każdej oktawy wykonywane są następujące operacje:

- wygenerowanie szumu Perlina,
- uwzględnienie amplitudy,
- aktualizacja częstotliwości,
- aktualizacja amplitudy.

Cały proces odbywa się lokalnie w obrębie pojedynczego wątku GPU.

Oznacza to, że każdy wątek generuje własną wartość fBm niezależnie od pozostałych wątków.

4.7 Kernel CUDA

Najważniejszym elementem implementacji GPU jest kernel:

```
1 __global__  
2 void noiseKernel(...)
```

Kwalifikator:

```
1 __global__
```

oznacza funkcję uruchamianą przez procesor CPU, ale wykonywaną przez GPU.

Po uruchomieniu kernela każdy wątek wyznacza współrzędne przypisanego mu piksela:

```
1 int x = blockIdx.x * blockDim.x + threadIdx.x;  
2  
3 int y = blockIdx.y * blockDim.y + threadIdx.y;
```

gdzie:

- `threadIdx` określa pozycję wątku w bloku,
- `blockIdx` określa pozycję bloku w siatce,
- `blockDim` określa rozmiar bloku.

W rezultacie każdy wątek otrzymuje unikalne współrzędne piksela.

Następnie wykonywane jest sprawdzenie:

```
1 if(x >= width || y >= height)
2 {
3     return;
4 }
```

które zabezpiecza przed dostępem poza obszar obrazu.

Po wyznaczeniu współrzędnych obliczana jest wartość fBm:

```
1 float value =
2 fbm_device(
3     x * scale,
4     y * scale,
5     octaves,
6     persistence,
7     lacunarity);
```

Wynik jest następnie normalizowany do zakresu $[0, 1]$, konwertowany do skali szarości i zapisywany w pamięci globalnej GPU:

```
1 output[y * width + x] =
2 static_cast<unsigned char>(gray);
```

4.8 Konfiguracja siatki wątków

Przed uruchomieniem kernela definiowany jest rozmiar bloków:

```
1 dim3 block(16, 16);
```

Każdy blok zawiera więc:

$$16 \times 16 = 256 \quad (51)$$

wątków.

Następnie obliczana jest liczba bloków potrzebnych do pokrycia całego obrazu:

```
1 dim3 grid(
2     (width + block.x - 1)/block.x,
3     (height + block.y - 1)/block.y);
```

Tak skonstruowana siatka gwarantuje, że każdy piksel obrazu zostanie przypisany do co najmniej jednego wątku.

4.9 Uruchomienie kernela

Kernel uruchamiany jest instrukcją:

```
1 noiseKernel<<<grid, block>>>(
2     d_output,
3     width,
4     height,
5     scale,
6     octaves,
```

```
7     persistence ,  
8     lacunarity);
```

Składnia:

```
1 <<<grid, block>>>
```

jest charakterystyczna dla CUDA i określa konfigurację uruchamianych bloków oraz wątków.

Po uruchomieniu GPU wykonuje równolegle wszystkie instancje kernela, a każda z nich generuje wartość szumu dla jednego piksela obrazu.

5 Optymalizacja implementacji CUDA

Poza podstawową implementacją dla GPU, przygotowano również wersję zoptymalizowaną, której celem było zmniejszenie ułatwienie kompilatorowi CUDA generowania bardziej efektywnego kodu maszynowego.

Optymalizacja nie zmienia działania algorytmu. Zarówno wersja podstawowa, jak i zoptymalizowana generują identyczne wartości szumu. Zmodyfikowany został jedynie sposób realizacji części operacji.

5.1 Zakres wprowadzonych optymalizacji

W stosunku do wersji podstawowej zastosowano następujące usprawnienia:

- wymuszenie rozwijania funkcji (`forceinline`),
- zastąpienie funkcji `floorf()` instrukcją sprzętową (`float2int`),
- wykorzystanie kwalifikatora (`restrict`),

5.2 Eliminacja narzutu wywołań funkcji

W wersji podstawowej funkcje:

```
1 __device__ float perlin_device(...)  
2 __device__ float fbm_device(...)
```

są wywoływane w standardowy sposób.

Każde wywołanie funkcji powoduje wykonanie dodatkowych operacji związanych z:

- przekazaniem argumentów,
- zapisaniem adresu powrotu,
- przejściem do kodu funkcji,
- powrotem do miejsca wywołania.

Dla pojedynczego wywołania koszt ten jest niewielki, jednak podczas generowania obrazu wykonywane są miliony takich operacji.

W wersji zoptymalizowanej zastosowano kwalifikator:

```
1 __forceinline__
```

przykładowo:

```
1 __device__ __forceinline__  
2 float perlin_opt(float x, float y)
```

oraz

```
1 __device__ __forceinline__  
2 float fbm_opt(...)
```

Kwalifikator ten wymusza na kompilatorze rozwinięcie funkcji bezpośrednio w miejscu wywołania.

Przykładowo zamiast:

```
1 value = fbm_opt(...);
```

kompilator może umieścić bezpośrednio instrukcje znajdujące się wewnątrz funkcji.

Pozwala to wyeliminować koszt związany z wywołaniem funkcji oraz często umożliwia dalsze optymalizacje wykonywane automatycznie przez kompilator.

5.3 Optymalizacja obliczania części całkowitej współrzędnych

Jedną z najczęściej wykonywanych operacji w algorytmie jest wyznaczenie współrzędnych komórki siatki.

W wersji podstawowej stosowano:

```
1 int X = ((int)floorf(x)) & 255;  
2 int Y = ((int)floorf(y)) & 255;
```

oraz:

```
1 x -= floorf(x);  
2 y -= floorf(y);
```

Funkcja `floorf()` zwraca największą liczbę całkowitą nie większą od podanej wartości.

Mimo że jest to rozwiązanie poprawne, wywołanie funkcji matematycznej generuje dodatkowy koszt obliczeniowy.

W wersji zoptymalizowanej zastosowano instrukcję:

```
1 __float2int_rd()
```

Przykład:

```
1 int X = __float2int_rd(x) & 255;  
2 int Y = __float2int_rd(y) & 255;
```

Instrukcja ta wykorzystuje sprzętowy mechanizm konwersji liczb zmiennoprzecinkowych na liczby całkowite z zaokrągleniem w dół.

W praktyce daje ten sam rezultat co:

```
1 (int) floorf(x)
```

jednak jest realizowana bezpośrednio przez jednostki obliczeniowe GPU i wymaga mniejszej liczby instrukcji.

Analogicznie wyznaczana jest część ułamkowa współrzędnych:

```
1 x -= (float) __float2int_rd(x);  
2 y -= (float) __float2int_rd(y);
```

5.4 Zastosowanie kwalifikatora restrict

W wersji zoptymalizowanej kernel został zadeklarowany jako:

```
1 __global__  
2 void noiseKernelOptimized(  
3     unsigned char* __restrict__ output,  
4     ...)
```

Kwalifikator:

```
1 __restrict__
```

informuje kompilator, że wskaźnik `output` nie współdzieli pamięci z innymi wskaźnikami wykorzystywanymi przez funkcję.

Dzięki temu kompilator może bezpiecznie wykonywać optymalizacje związane z dostępem do pamięci.

Bez takiej informacji kompilator musi zakładać możliwość nakładania się różnych obszarów pamięci, co często ogranicza możliwości optymalizacji.

W implementacji generowania szumu wykorzystywany jest tylko jeden bufor wyjściowy, dlatego zastosowanie `restrict` jest bezpieczne.

5.5 Porównanie funkcji `perlin_device()` i `perlin_opt()`

Najważniejsza różnica pomiędzy obiema implementacjami dotyczy sposobu wyznaczania współrzędnych komórki.

Wersja podstawowa:

```
1 int X = ((int) floorf(x)) & 255;  
2 int Y = ((int) floorf(y)) & 255;  
3  
4 x -= floorf(x);  
5 y -= floorf(y);
```

Wersja zoptymalizowana:

```
1 int X = __float2int_rd(x) & 255;  
2 int Y = __float2int_rd(y) & 255;  
3  
4 x -= (float) __float2int_rd(x);  
5 y -= (float) __float2int_rd(y);
```

Pozostała część funkcji pozostaje praktycznie niezmieniona.

Optymalizacja nie wpływa na jakość generowanego szumu, lecz jedynie zmniejsza koszt wykonania poszczególnych obliczeń.

5.6 Porównanie funkcji `fbm_device()` i `fbm_opt()`

Funkcje realizują identyczny algorytm sumowania kolejnych oktaw.

Jedyną różnicą jest zastosowanie kwalifikatora:

```
1 __forceinline__
```

który pozwala kompilatorowi włączyć kod funkcji bezpośrednio do kernela.

Dzięki temu eliminowane są dodatkowe wywołania funkcji wykonywane dla każdej oktawy i każdego piksela obrazu.

Przy dużych rozdzielczościach liczba takich wywołań jest bardzo duża, dlatego nawet niewielkie oszczędności przekładają się na zauważalny wzrost wydajności.

5.7 Wpływ optymalizacji na wydajność

Wprowadzone usprawnienia należą do grupy optymalizacji niskopoziomowych. Nie zmieniają organizacji obliczeń ani algorytmu generowania szumu, lecz zmniejszają liczbę instrukcji wykonywanych przez każdy wątek. Ponieważ każda próbka szumu wymaga wielokrotnego wykonania tych samych operacji, nawet niewielkie skrócenie czasu pojedynczego obliczenia może przynieść zauważalne przyspieszenie podczas generowania całego obrazu.

6 Obsługa benchmarków oraz zapisu wyników

Oprócz implementacji algorytmu szumu Perlina projekt zawiera moduły odpowiedzialne za przeprowadzanie testów wydajnościowych, zapisywanie wygenerowanych obrazów oraz eksport wyników do plików CSV. Dzięki temu możliwe jest bezpośrednie porównanie wydajności implementacji CPU, GPU oraz GPU_OPT dla różnych parametrów generowania szumu.

6.1 Program główny

Punkt wejścia aplikacji stanowi funkcja `main()`, której zadaniem jest pobranie od użytkownika podstawowych parametrów testu oraz uruchomienie procedury benchmarkowej.

Użytkownik może określić:

- czy zapisywać wygenerowane obrazy,
- czy zapisywać wyniki do pliku CSV,
- czy wykonać fazę rozgrzewki (*warm-up*),
- liczbę iteracji benchmarku.

Dodatkowo definiowane są parametry szumu Perlina:

- skala (`scale`),
- liczba oktav (`octaves`),
- współczynnik zaniku amplitudy (`persistence`),
- współczynnik wzrostu częstotliwości (`lacunarity`).

W programie zdefiniowano również zestaw testowanych rozdzielczości:

$$256x256, 512x512, 1024x1024, 2048x2048. \quad (52)$$

Po skonfigurowaniu parametrów wywoływana jest funkcja:

```
1 runBenchmark(...);
```

odpowiedzialna za przeprowadzenie wszystkich pomiarów.

6.2 Moduł benchmarków

Główna logika testów została zaimplementowana w funkcji `runBenchmark()`.

Przed rozpoczęciem właściwych pomiarów opcjonalnie wykonywana jest faza warm-up. Polega ona na kilkukrotnym uruchomieniu wszystkich implementacji dla niewielkiej rozdzielczości obrazu.

Celem tego etapu jest :

- zapobieganie przekłamania pierwszego pomiaru,
- inicjalizacja pamięci i struktur danych,
- eliminacja kosztu pierwszego uruchomienia kernela CUDA,
- uzyskanie bardziej stabilnych wyników pomiarów.

Następnie dla każdej testowanej rozdzielczości tworzona jest osobna tablica obrazu dla implementacji CPU, GPU oraz GPU_OPT.

W każdej iteracji wykonywane są kolejno trzy pomiary:

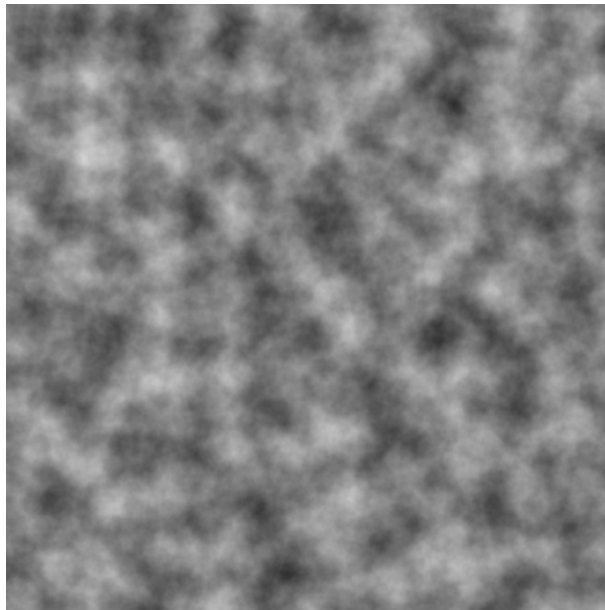
```
1 generateNoiseCPU(...)
2 generateNoiseGPU(...)
3 generateNoiseGPUOptimized(...)
```

Uzyskane czasy są sumowane, a po zakończeniu wszystkich iteracji obliczana jest średnia wartość czasu wykonania. Końcowe wyniki prezentowane są w konsoli programu lub opcjonalnie w pliku csv.

6.3 Zapis obrazów

Wygenerowane obrazy mogą zostać zapisane do plików w formacie PGM (*Portable Gray Map*).

Format ten przechowuje obraz w postaci macierzy wartości jasności pikseli z zakresu:



Rysunek 1: Przykład wygenerowanego szumu perlina

$$0 \leq I \leq 255. \quad (53)$$

Zapis realizowany jest przez funkcję:

```
1 savePGM(...)
```

Na początku pliku zapisywany jest nagłówek formatu PGM:

```
1 P2
2 width height
3 255
```

Po nagłówku zapisywane są wartości wszystkich pikseli obrazu.

Dla każdej rozdzielczości tworzone są trzy pliki:

- XXX_cpu.pgm,
- XXX_gpu.pgm,
- XXX_gpu_opt.pgm,

gdzie XXX oznacza rozdzielczość obrazu.

6.4 Eksport wyników do pliku CSV

W celu późniejszej analizy wyników zaimplementowano moduł eksportu danych do formatu CSV.

Na początku działania programu tworzony jest plik:

```
1 benchmark.csv
```

z nagłówkiem:

```
1 width;height;implementation;  
2 iteration;time
```

Podczas każdej iteracji benchmarku wykonywany jest zapis aktualnych wyników przy pomocy funkcji:

```
1 appendBenchmarkResult(...)
```

Każdy rekord zawiera:

- szerokość obrazu,
- wysokość obrazu,
- nazwę implementacji,
- numer iteracji,
- zmierzony czas wykonania.

Dzięki zapisowi wyników w formacie CSV możliwe jest późniejsze wykorzystanie zewnętrznych narzędzi analitycznych, takich jak Python, Excel czy MATLAB, do tworzenia wykresów i przeprowadzania dalszej analizy wydajności.

7 Analiza wyników i wnioski

Przeprowadzone testy porównują czas generowania szumu Perlina dla trzech implementacji:

- CPU – implementacja sekwencyjna w języku C++,
- GPU – podstawowa implementacja CUDA,
- GPU_OPT – zoptymalizowana implementacja CUDA.

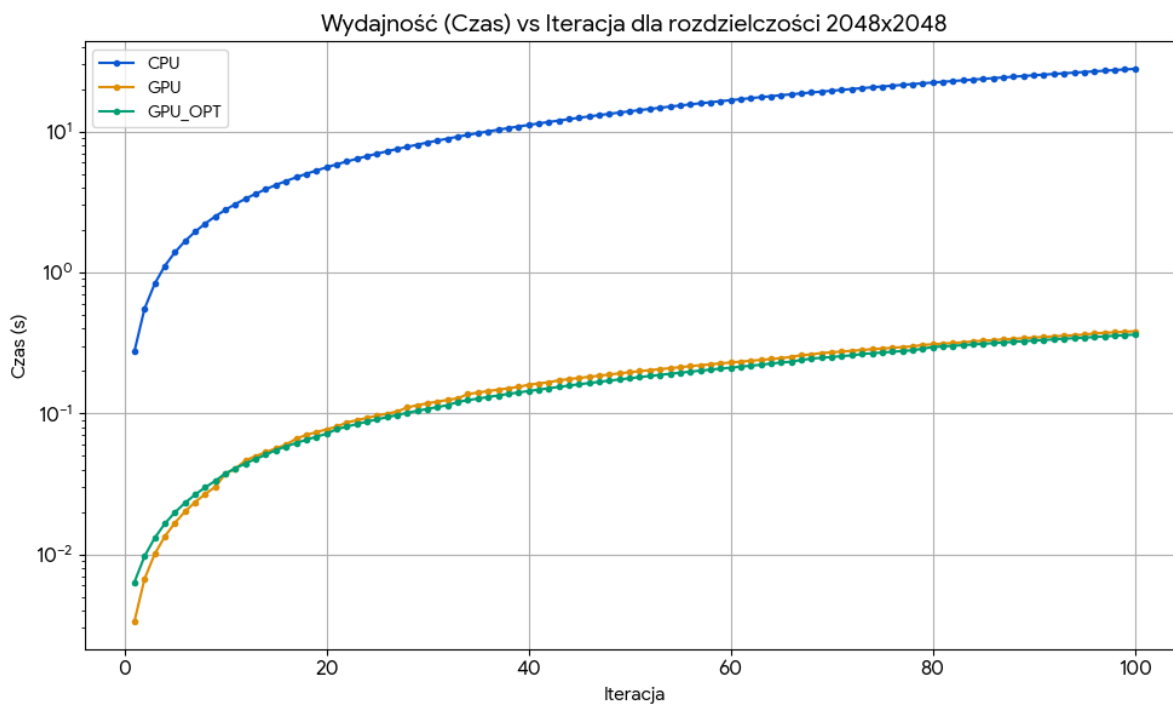
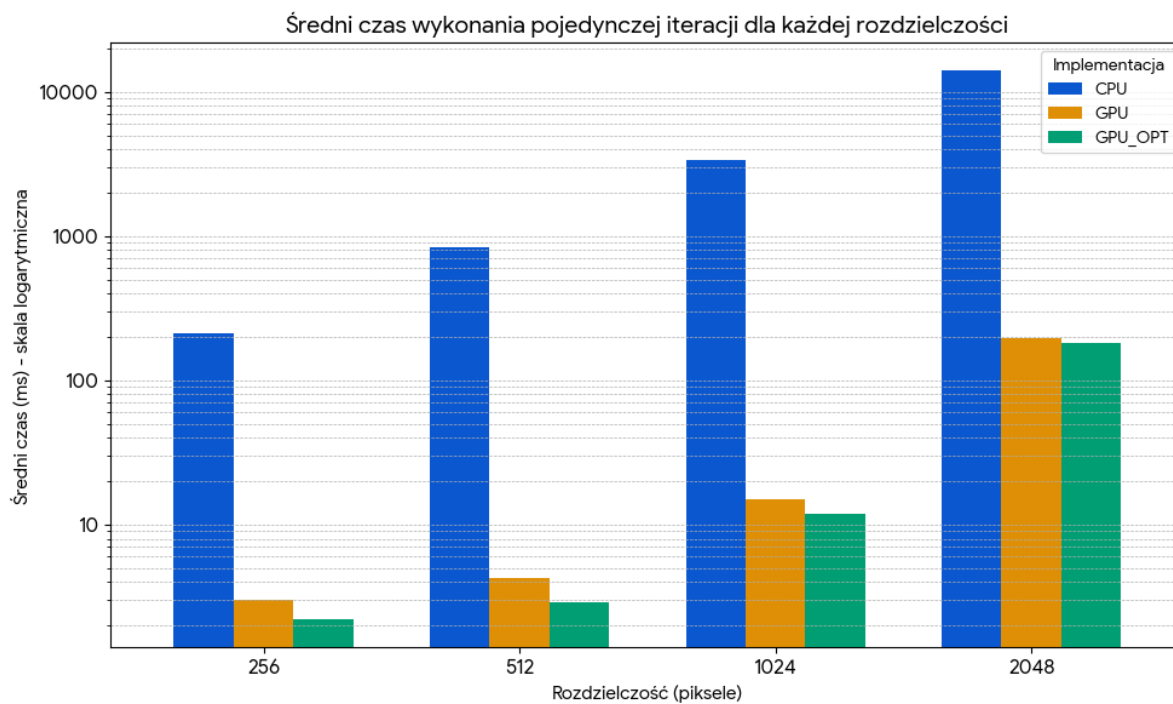
Pomiary wykonano dla rozdzielczości 256×256 , 512×512 , 1024×1024 oraz 2048×2048 , zwiększając liczbę iteracji od 1 do 100.

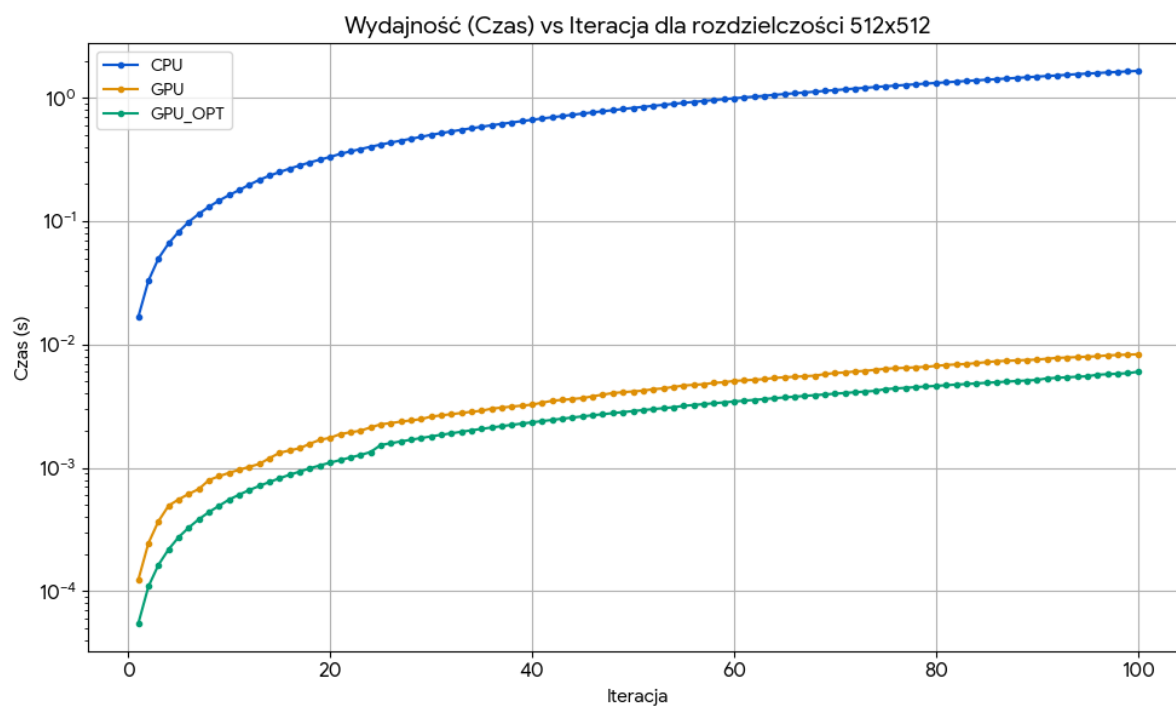
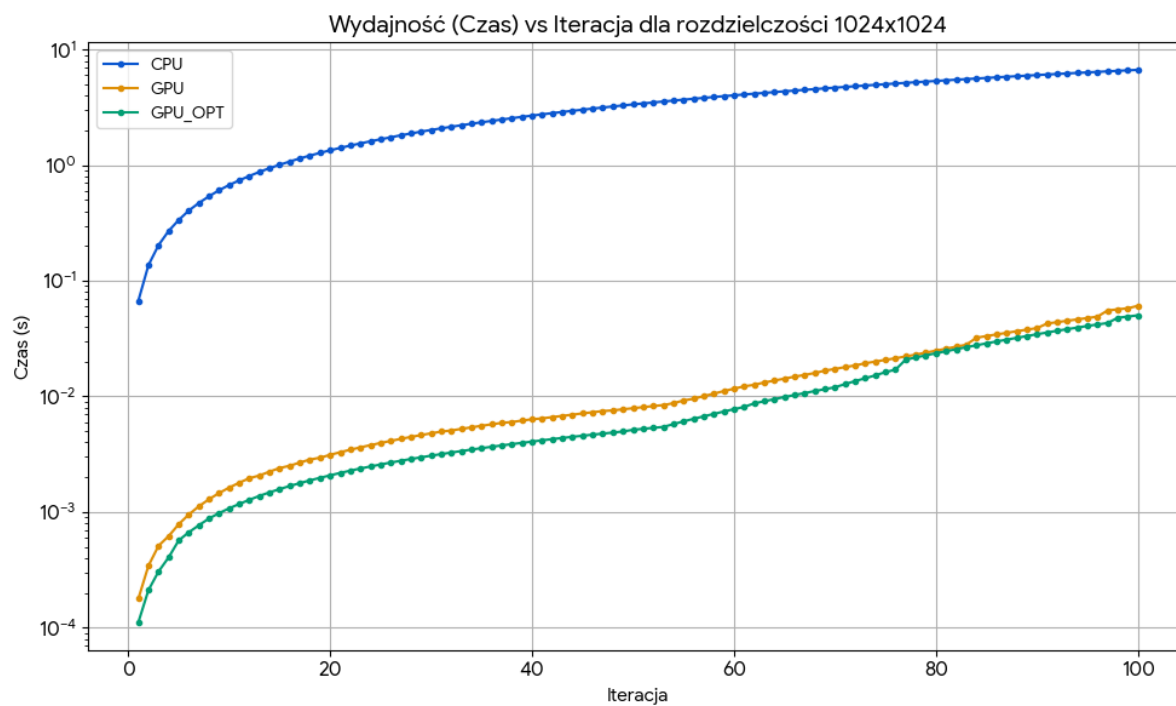
Na wszystkich wykresach można zaobserwować wzrost czasu wykonania wraz ze wzrostem liczby oktaw. Jest to zgodne z teorią działania algorytmu fBm, ponieważ każda dodatkowa oktawa wymaga wykonania kolejnego wywołania funkcji szumu Perlina.

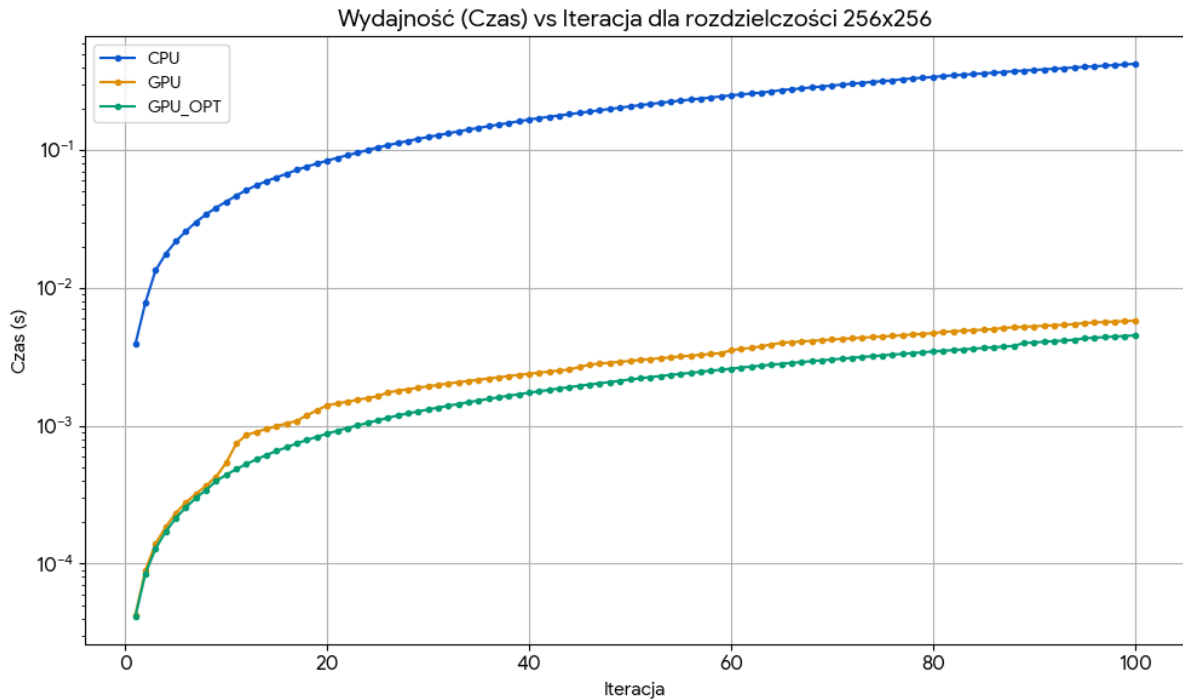
Implementacja CPU charakteryzuje się największym czasem wykonania dla wszystkich badanych rozdzielczości. Wraz ze wzrostem liczby pikseli różnica pomiędzy CPU a GPU staje się coraz większa, co wynika z sekwencyjnego charakteru obliczeń wykonywanych przez procesor.

Implementacja GPU osiąga znacznie lepsze wyniki dzięki równoległemu przetwarzaniu pikseli przez tysiące wątków. Dla wszystkich rozdzielczości czas wykonania jest wielokrotnie krótszy niż w przypadku wersji CPU.

Wersja GPU_OPT osiąga najlepsze rezultaty niemal w całym zakresie testów. Widoczna przewaga nad wersją podstawową jest szczególnie zauważalna dla małych i średnich rozdzielczości, gdzie narzut związany z pojedynczymi instrukcjami ma większy wpływ na







całkowity czas wykonania. W przypadku największej rozdzielczości (2048×2048) różnica pomiędzy obiema wersjami GPU maleje, ponieważ dominującym czynnikiem staje się liczba wykonywanych operacji oraz przepustowość pamięci.

7.1 Wnioski

Na podstawie przeprowadzonych benchmarków można sformułować następujące wnioski:

1. Implementacja GPU znacząco przyspiesza generowanie szumu Perlina względem implementacji CPU dla wszystkich badanych rozdzielczości.
2. Wraz ze wzrostem rozdzielczości przewaga GPU nad CPU rośnie, co potwierdza wysoką skalowalność algorytmu w środowisku równoległym.
3. Zastosowane optymalizacje CUDA (`__forceinline__`, `__float2int_rd()`, `__restrict__`) przynoszą zauważalne skrócenie czasu wykonania bez wpływu na jakość generowanego szumu.
4. Największe korzyści z optymalizacji widoczne są dla małych i średnich rozdzielczości, gdzie redukcja narzutu instrukcji ma największe znaczenie.
5. Dla bardzo dużych obrazów różnica pomiędzy wersją GPU i GPU_OPT maleje, co wskazuje, że ograniczeniem staje się głównie przepustowość pamięci oraz liczba wykonywanych operacji.
6. Algorytm szumu Perlina należy do problemów bardzo dobrze poddających się równoległemu przetwarzaniu, ponieważ obliczenia poszczególnych pikseli są od siebie całkowicie niezależne.
7. Uzyskane wyniki potwierdzają, że wykorzystanie procesora graficznego jest skuteczną metodą przyspieszenia procedur generowania szumu proceduralnego wyko-

rzystywanych w grafice komputerowej, generowaniu terenów, symulacjach oraz systemach wizualizacji.

Podsumowując, implementacja CUDA pozwoliła na uzyskanie wielokrotnego przyspieszenia względem rozwiązania procesorowego, natomiast dodatkowe optymalizacje niskopoziomowe umożliwiły dalszą poprawę wydajności bez konieczności modyfikacji samego algorytmu szumu Perlina.